MATLAB Code:

```
% Collecting, filtering and plotting ECG signals

clear;
clc;
close all;

% Enter your Arduino COM port here
arduino = serialport("COM3", 115200);

% Allow the Arduino to restart
pause(2);

% Clear any old data
flush(arduino);

% Number of samples taken every second
sampleRate = 250;

% Collect data for 10 seconds
recordingTime = 10;

% Calculate the total number of samples
totalSamples = sampleRate * recordingTime;
```

```
% Create empty arrays to store the data
ECG = zeros(totalSamples,1);
LOminus = zeros(totalSamples,1);
LOplus = zeros(totalSamples,1);

disp("Recording ECG signal...");

for i = 1:totalSamples

    % Read one line from the Arduino
    data = readline(arduino);

    % Separate the three values
    values = sscanf(data,'%f,%f,%f');

    % Check that all three values have arrived
    if length(values) == 3

        ECG(i) = values(1);
        LOminus(i) = values(2);
        LOplus(i) = values(3);

    else

        % Ignore an invalid measurement
        ECG(i) = NaN;
```

```
        LOminus(i) = NaN;
        LOplus(i) = NaN;

    end

end

% Close the serial connection
clear arduino;

disp("Recording complete!");



% Check whether either electrode disconnected
if any(LOminus == 1) || any(LOplus == 1)

    warning("Electrode disconnection detected! Check the ECG electrodes.");

else

    disp("Both electrode connections appear to be stable.");

end



% Create a time value for every ECG measurement
```

```
time = (0:totalSamples-1) / sampleRate;



% Find valid ECG measurements
validData = ~isnan(ECG);

% Keep only valid measurements
ECG = ECG(validData);
time = time(validData);

% Update the total number of valid samples
totalSamples = length(ECG);

% Set the lower and upper filter frequencies
lowFrequency = 0.5;
highFrequency = 40;

% Create a fourth-order Butterworth bandpass filter
[b,a] = butter(4, ...
   [lowFrequency highFrequency]/(sampleRate/2), ...
   'bandpass');



% Remove low-frequency and high-frequency noise
filteredECG = filtfilt(b,a,ECG);
```

```
figure;

subplot(2,1,1);

plot(time,ECG);

title('Original ECG Signal');

xlabel('Time (seconds)');
ylabel('ADC Reading');

grid on;



subplot(2,1,2);

plot(time,filteredECG);

title('Filtered ECG Signal');

xlabel('Time (seconds)');
ylabel('Amplitude (ADC units)');

grid on;
```

```
% Estimate the minimum distance between heartbeats
minimumDistance = round(0.3 * sampleRate);

% Set the minimum peak prominence
minimumProminence = 0.5 * std(filteredECG);

% Find the R peaks
[peakValues,peakLocations] = findpeaks(filteredECG, ...
   'MinPeakDistance',minimumDistance, ...
   'MinPeakProminence',minimumProminence);

% Convert peak locations into time
peakTimes = time(peakLocations);

% Calculate the time between consecutive R peaks
RRintervals = diff(peakTimes);

% Calculate heart rate in beats per minute
if length(RRintervals) >= 1

   heartRate = 60 ./ RRintervals;

   % Calculate average heart rate
   averageHeartRate = mean(heartRate);
```

```
    disp("Average heart rate:");
    disp(averageHeartRate);

else

    averageHeartRate = NaN;

    warning("Not enough R peaks detected to calculate heart rate.");

end

figure;

plot(time,filteredECG);
hold on;

plot(peakTimes,peakValues,'ro','MarkerFaceColor','r');

title('Filtered ECG with Detected R Peaks');

xlabel('Time (seconds)');
ylabel('Amplitude (ADC units)');

legend('Filtered ECG','R Peaks');

grid on;
```

```
figure;

% Display the first three seconds
plot(time,filteredECG);

xlim([0 min(3,time(end))]);

title('Detailed ECG Waveform - PQRST');

xlabel('Time (seconds)');
ylabel('Amplitude (ADC units)');

grid on;

disp("ECG processing finished!");
```